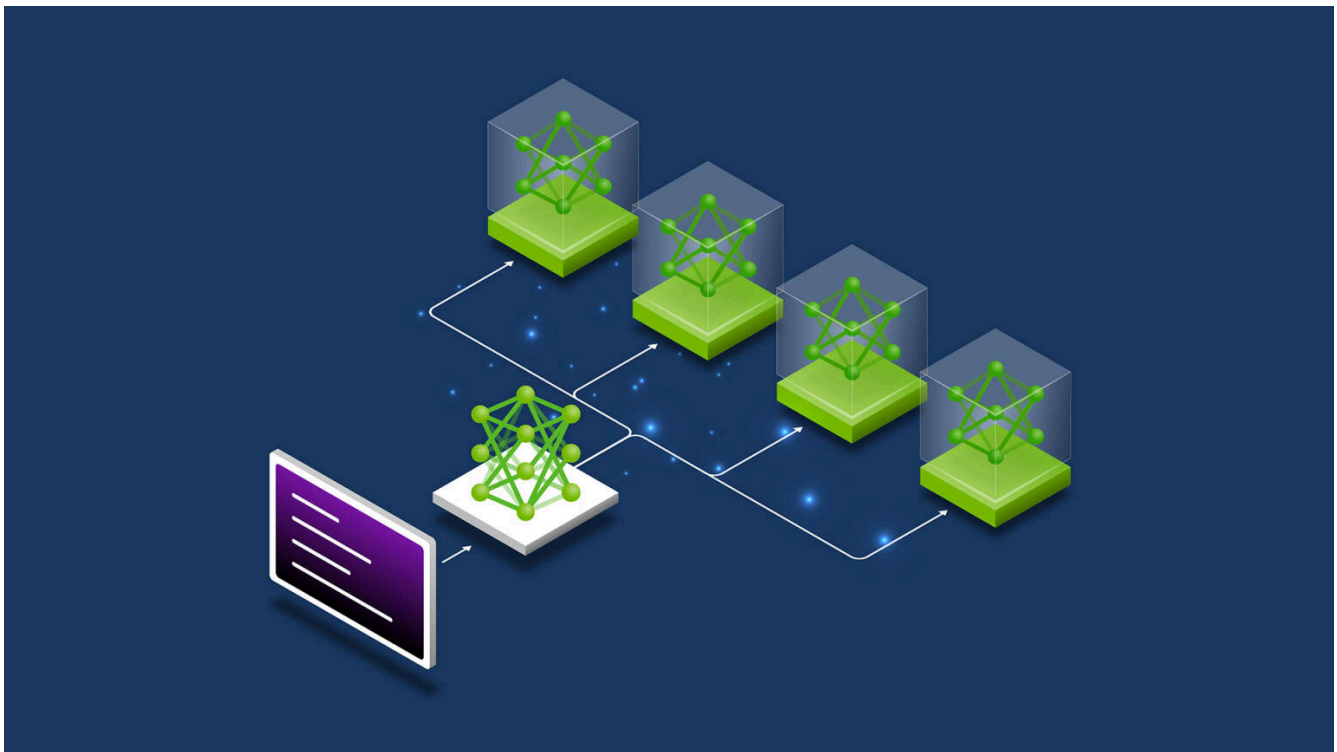




Как запускать изолированные кластеры Kubernetes для арендаторов на общей инфраструктуре с графическими процессорами



3 августа 2026 года

+5 Нравится

Обсудить (0)

Авторы: [Экин Карабулут](#) и [Сайям Патхак](#)



Сгенерированное искусственным интеллектом резюме





несколькими командами, по мере роста их числа затраты на координацию увеличиваются. Среди проблем — конфликтующие версии CRD, дублирование RBAC и отсутствие четкого способа распределения ресурсов графического процессора между командами. При определенном масштабе команды могут начать требовать собственные кластеры только для того, чтобы вернуть себе независимость.

В этом посте представлен шаблон, который позволяет сохранить автономность команд без разделения оборудования. Это решение включает в себя единый кластер плоскости управления с пулом графических процессоров, совместное использование графических процессоров с квотами для каждой команды и изолированную плоскость управления Kubernetes для каждой команды, включающую в себя сервер API, контроллер, хранилище данных, синхронизатор и планировщик. Этого можно добиться с помощью двух инструментов с открытым исходным кодом: [KAI Scheduler](#) и [vCluster](#).

Следуйте инструкциям, и у вас будет три команды, использующие настоящие GPU-модули Kubernetes в собственных кластерах, при этом все они будут использовать один физический GPU. Вы также сможете убедиться, что каждая команда видит только свои рабочие нагрузки.

Предварительные требования к учебному пособию и примечания

Чтобы процесс был воспроизводимым для пользователей с ограниченными ресурсами, в этом руководстве используется кластер с одним [графическим процессором NVIDIA L40S](#) и тремя командами, которые делят его между собой. Это позволяет легко увидеть и опробовать все этапы. Процесс работает так же в более крупном кластере с сотнями графических процессоров и десятками команд. Вы можете масштабировать пул узлов, иерархию очередей и количество клиентских кластеров.

[KAI Scheduler](#) — это надежный, эффективный и масштабируемый планировщик Kubernetes с учетом топологии, специально разработанный для оптимизации распределения ресурсов графических процессоров при выполнении задач искусственного интеллекта. Он предназначен для управления крупномасштабными кластерами графических процессоров, включающими тысячи узлов, и обеспечения высокой пропускной способности при выполнении рабочих нагрузок. С помощью KAI Scheduler вы можете динамически распределять ресурсы графических процессоров между рабочими нагрузками. Он может работать параллельно с планировщиком kube-scheduler по умолчанию. Любой под с



Платформа vCluster Kubernetes предоставляет полностью изолированные кластеры для арендаторов в вашей инфраструктуре или непосредственно на «голом железе». Каждый кластер для арендаторов получает собственный API-сервер, пользовательские определения ресурсов (Custom Resource Definitions, CRD) и управление доступом на основе ролей (Role-Based Access Control, RBAC), неотличимые от выделенного кластера Kubernetes, при этом используются одни и те же базовые узлы и оборудование. Виртуализированная плоскость управления невидима для арендаторов: нет общих узлов плоскости управления, нет агентов внутри кластера и нет латерального пути между средами. Поэтому vCluster идеально подходит для инфраструктуры с графическими процессорами, где командам нужен собственный чистый кластер без разделения оборудования.

В этом руководстве используется модель vCluster с общими узлами, поэтому команды совместно используют узел с графическим процессором, но при этом у каждой есть собственная изолированная плоскость управления. Это оптимальный вариант для доверенных внутренних команд. Для недоверенных клиентов, которым требуется разделение на уровне узлов, сетей и хранилищ, можно использовать частные узлы vCluster.

В примере, приведенном в этом посте, участвуют три команды: команда по обработке естественного языка, команда по машинному зрению и команда по разработке рекомендательной системы. Команда по обработке естественного языка хочет установить собственные CRD. Команда по машинному зрению хочет, чтобы администратор кластера отладил планирование. Команда по разработке рекомендательной системы использует другую версию Kubeflow. Никто не хочет делиться контекстом kubectl и случайно ломать чужие среды.

При использовании vCluster у каждой команды есть собственная изолированная плоскость управления, система управления доступом на основе ролей, пространства имен и CRD. У каждой команды может быть доступ администратора кластера. При этом все кластеры арендаторов используют одни и те же узлы и графические процессоры.

Демонстрационная среда

Эта демонстрация выполняется на инстансе графического процессора NVIDIA Brev на Nebius со следующими характеристиками:

- Один NVIDIA L40S, 40 виртуальных ЦП, 160 ГБ ОЗУ, диск на 256 ГБ (48 ГБ видеопамати)
- Ubuntu 24.04.4 LTS
- MicroK8s v1.36.2 — Kubernetes был предварительно настроен Brev, включая дополнение MicroK8s gpu, которое предварительно устанавливает NVIDIA GPU Operator в пространство имен gpu-operator-resources



Note: For different setups, the cluster-creation and GPU Operator install steps will differ on GKE/EKS/AKS/vanilla k8s/k3s. Step 3 (KAI Scheduler) onward is identical on any Kubernetes that has the NVIDIA GPU Operator running with Container Device Interface (CDI) enabled.

Figure 1. KAI Scheduler and vCluster architecture on a shared Nebius host cluster

Step 1: Local tooling

Install standalone `kubectl` and `helm` to avoid prefixing every command with `microk8s`. Then wire up a `kubeconfig` and pin MicroK8s so `snap` doesn't auto-upgrade the control plane during the demo.

```
sudo snap refresh --hold microk8s

sudo snap install kubectl --classic --channel=1.35/stable
sudo snap install helm --classic

mkdir -p ~/.kube
sudo microk8s config > ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
chmod 600 ~/.kube/config
```

Next, make sure the required MicroK8s addons are enabled:

```
microk8s enable dns
microk8s enable hostpath-storage    # vCluster needs PVCs
```



```
kubectl get nodes -o wide  
kubectl get storageclass
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
brev-8dq0cch1j	Ready	<none>	57d	v1.36.2	10.0.0.20	<none>	Ubuntu 24.04

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
microk8s-hostpath (default)	microk8s.io/hostpath	Delete	WaitForFirstConsumer

Step 2: Add the Helm repo

Add the NVIDIA Helm repo:

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia  
helm repo update
```

Step 3: Confirm the GPU Operator

```
kubectl get pods -n gpu-operator-resources
```

NAME	READY	STATUS	RESTARTS
gpu-feature-discovery-wpcjm	1/1	Running	0
gpu-operator-57d75775c8-npjzz	1/1	Running	0
gpu-operator-node-feature-discovery-...	1/1	Running	0
nvidia-container-toolkit-daemonset-ft4hb	1/1	Running	0
nvidia-cuda-validator-8vqdv	0/1	Completed	0
nvidia-device-plugin-daemonset-l6fj6	1/1	Running	0
nvidia-operator-validator-cdnmj	1/1	Running	0

If you're using an older GPU Operator, upgrade in place to 26.3.x:

```
helm upgrade gpu-operator nvidia/gpu-operator \\\n  -n gpu-operator-resources \\\n  --version v26.3.3 \\\n  --reset-then-reuse-values
```

If needed, install [NVIDIA GPU Operator](#) directly:

```
helm install gpu-operator nvidia/gpu-operator \\\n  -n gpu-operator-resources --create-namespace \\\n  --version v26.3.3 \\\n  --set driver.enabled=false \\\n  --set operator.defaultRuntime=containerd \\\n  --set toolkit.env[0].name=CONTAINERD_CONFIG \\\n  --set toolkit.env[0].value=/var/snap/microk8s/current/args/containerd.toml \\\n
```



```
--set-string COOINTECH[2].value=1
```

Step 4: Install KAI Scheduler

Next, install the KAI Scheduler:

```
helm upgrade -i kai-scheduler \
  oci://ghcr.io/kai-scheduler/kai-scheduler/kai-scheduler \
  -n kai-scheduler --create-namespace \
  --version v0.16.4 \
  --set "global.gpuSharing=true"
```

Then verify:

```
kubectl get pods -n kai-scheduler
```

NAME	READY	STATUS	RESTARTS	AGE
admission-57556f949-sp98t	1/1	Running	0	47s
binder-66785d8dd9-9frgk	1/1	Running	0	46s
kai-operator-6fdf595c4d-292d7	1/1	Running	0	52s
kai-scheduler-default-6bb667b767-vq2q4	1/1	Running	0	46s
pod-grouper-84dfc7759b-v5qtb	1/1	Running	0	47s
podgroup-controller-5878f48dbb-v7wcn	1/1	Running	0	47s
queue-controller-7796bb8984-hdn5r	1/1	Running	0	46s

Step 5: Define team queues

KAI Scheduler uses a Queue CRD to model an org → team hierarchy. This step involves creating one parent (ml-org, with a total budget of one GPU) and three child queues. Each is guaranteed 0.33 of the GPU and allowed to increase to the full GPU when others are idle.

Save the following as create-queues.yaml:

```
apiVersion: scheduling.run.ai/v2
kind: Queue
metadata:
  name: ml-org
spec:
  resources:
    gpu: { quota: 1, limit: -1, overQuotaWeight: 1 }
---
apiVersion: scheduling.run.ai/v2
kind: Queue
metadata:
  name: team-nlp
```



```
resources:
  gpu: { quota: 0.33, limit: 1, overQuotaWeight: 1 }
---
apiVersion: scheduling.run.ai/v2
kind: Queue
metadata:
  name: team-vision
spec:
  parentQueue: ml-org
  priority: 100
  resources:
    gpu: { quota: 0.33, limit: 1, overQuotaWeight: 1 }
---
apiVersion: scheduling.run.ai/v2
kind: Queue
metadata:
  name: team-recommender
spec:
  parentQueue: ml-org
  priority: 100
  resources:
    gpu: { quota: 0.33, limit: 1, overQuotaWeight: 1 }
```

Here, `quota` is the guaranteed minimum, `limit` is the maximum allowed, and `overQuotaWeight` controls how surplus is split.

Next, apply and list:

```
kubectl apply -f create-queues.yaml
kubectl get queues
```

```
queue.scheduling.run.ai/ml-org created
queue.scheduling.run.ai/team-nlp created
queue.scheduling.run.ai/team-vision created
queue.scheduling.run.ai/team-recommender created
```

NAME	PRIORITY	PARENT	CHILDREN
default-parent-queue			["default-queue"]
default-queue		default-parent-queue	
ml-org			["team-nlp", "team-vision", "team-re
team-nlp	100	ml-org	
team-recommender	100	ml-org	
team-vision	100	ml-org	

Note that `default-parent-queue` and `default-queue` are created automatically by KAI Scheduler on first install. They are the fallback queue for any pod that doesn't specify one.

Step 6: Spin up a vCluster per team



```
curl -L -o vcluster https://github.com/loft-sh/vcluster/releases/latest/download/vcluster-  
sudo install -m 755 vcluster /usr/local/bin/vcluster  
rm vcluster  
vcluster --version
```

```
vcluster version 0.35.1
```

Define the vCluster config. The critical setting is `setOwner: false`. The KAI Scheduler pod-grouper walks ownership chains (Job → Pod, Deployment → ReplicaSet → Pod) to auto-group workloads. Disabling vCluster owner rewriting allows KAI Scheduler to see the real hierarchy.

```
cat > vcluster.yaml <<'EOF'  
experimental:  
  syncSettings:  
    setOwner: false  
  
sync:  
  fromHost:  
    nodes:  
      enabled: true  
      selector:  
        all: true  
EOF
```

Then create one vCluster per team:

```
vcluster create team-nlp --values vcluster.yaml --connect=false  
vcluster create team-vision --values vcluster.yaml --connect=false  
vcluster create team-recommender --values vcluster.yaml --connect=false
```

Verify that all three vClusters are running:

```
vcluster list
```

NAME	NAMESPACE	STATUS	VERSION	CONNECTED	AGE
team-nlp	vcluster-team-nlp	Running	0.35.1		102s
team-recommender	vcluster-team-recommender	Running	0.35.1		88s
team-vision	vcluster-team-vision	Running	0.35.1		93s

Each team sees the real nodes, including the GPU node:

```
vcluster connect team-nlp -- kubectl get nodes
```

```
19:09:22 done vCluster is up and running  
NAME          STATUS    ROLES    AGE    VERSION  
brev-8dq0cch1j Ready    <none>   6m20s  v1.36.2
```




team

Each team deploys their GPU workload from their own vCluster. The pod spec is simple, with three fields telling KAI Scheduler what to do:

```
vcluster connect team-nlp -- kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: nlp-sentiment-model
  labels:
    kai.scheduler/queue: team-nlp # Which team
  annotations:
    gpu-fraction: "0.33"          # How much GPU
spec:
  schedulerName: kai-scheduler.  # Use KAI, not default
  tolerations:
    - key: nvidia.com/gpu
      operator: Exists
      effect: NoSchedule
  containers:
    - name: nlp-inference
      image: nvidia/cuda:12.4.0-base-ubuntu22.04
      command: ["bash", "-c", "nvidia-smi; sleep infinity"]
  nodeSelector:
    nvidia.com/gpu.present: "true"
EOF
```

```
19:13:34 done vCluster is up and running
pod/nlp-sentiment-model created
```

Do the same to deploy from each team, changing the queue name.

Now, verify:

NAMESPACE	NAME	READY	STATUS
vcluster-team-nlp	nlp-sentiment-model-x-default-x-team-nlp	1/1	Runnin
vcluster-team-recommender	recommender-model-x-default-x-team-recommender	1/1	Runnin
vcluster-team-vision	vision-classifier-model-x-default-x-team-vision	1/1	Runnin

Three pods, three different vcluster-team-* namespaces, all running on the same physical node brev-8dq0cch1j.

Each team sees only their own pod from inside their vCluster:



NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
nlp-sentiment-model	1/1	Running	0	3m11s	10.1.171.180	brev-8dq0cch1j	
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	
vision-classifier-model	1/1	Running	0	3m59s	10.1.171.145	brev-8dq0cch1j	
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NO
recommender-model	1/1	Running	0	2m45s	10.1.171.187	brev-8dq0cch1j	<n

Finally, the queue confirms the allocation. You can check all three at the same time:

```
kubectl describe queue team-nlp | grep -A4 Status
kubectl describe queue team-vision | grep -A4 Status
kubectl describe queue team-recommender | grep -A4 Status
```

You will see the same result for all:

```
Status:
  Allocated:
    nvidia.com/gpu: 330m
  Requested:
    nvidia.com/gpu: 330m
```

Note that KAI Scheduler handles scheduling—which pods land on which GPU and in what proportion. It does not enforce GPU memory isolation at the hardware level when GPU sharing is used. Applications need to respect their memory amount (for example, setting `-gpu-memory-utilization` in vLLM).

Under the hood, the GPU time-slices between CUDA contexts from each pod at kernel boundaries. For hard memory isolation on supported hardware, NVIDIA Multi-Instance GPU (MIG) provides hardware-level partitioning, which can be scheduled by KAI Scheduler as well.

Get started running isolated tenant Kubernetes clusters

KAI Scheduler decides fairly who gets GPU slices and schedules them as a group using GPU sharing and DRA Driver support, hierarchical queues with guaranteed quotas plus over-quota capabilities, gang scheduling, and topology awareness. Once AI workload scheduling is solved, teams want their own clusters. vCluster gives each team its own isolated Kubernetes control plane without the cost of separate infrastructure.



infrastructure.

Ready to get started? Check out [KAI Scheduler](#), [vCluster](#), and [NVIDIA GPU Operator](#) on GitHub.

Learn more about KAI Scheduler and vCluster integration at [KubeCon 2026 North America](#), November 9-12.

Discuss (0)

+5 Like

Tags

[Data Center / Cloud](#) | [Developer Tools & Techniques](#) | [General](#) | [Run:AI](#) | [Intermediate Technical](#) | [Tutorial](#) | [Kubernetes](#)

About the Authors

About Ekin Karabulut

Ekin Karabulut is a data scientist and developer advocate previously at Run:ai, now at NVIDIA, exploring the efficient usage of large models in different production scenarios. Previously she worked on privacy implications of federated learning, focused on distributed training techniques and got fascinated by inefficiencies in GPU usage in research and industry settings. She established the AI Infrastructure Club and is based in Munich, Germany.

[View all posts by Ekin Karabulut](#) >

About Saiyam Pathak

Saiyam Pathak is working as head of DevRel at vCluster. He is the founder of Kubesimplify, focusing on simplifying cloud-native and AI infrastructure. He is Kubecon co-chair and has worked on many facets of Kubernetes, including machine learning platforms, scaling, multi-cloud, and managed Kubernetes services. When not coding, Saiyam contributes to the community by writing blogs and organizing local meetups for Kubernetes and CNCF. He is also a CNCF TAG OpsRes Chair.

[View all posts by Saiyam Pathak](#) >

Comments



Streamline AI Infrastructure with NVIDIA Run:ai on Microsoft Azure

NVIDIA Open Sources Run:ai Scheduler to Foster Community Collaboration



Practical Tips for Preventing GPU Fragmentation for Volcano Scheduler

Improving GPU Utilization in Kubernetes



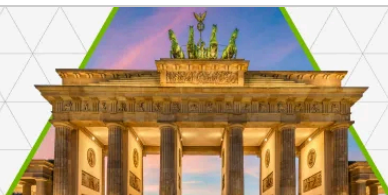
Maximizing NVIDIA DGX with Kubernetes

NVIDIA GTC Berlin

Registration Is Now Open

October 20–22

Register Early to Save



Sign up for NVIDIA News

Subscribe

Follow NVIDIA Developer

